

WEB-BASED LARGE PRIME VALIDATION SERVICE

P

- A web application for efficient validation of large integers
 - Project Presentation
- Algorithms • Number Theory • Web Technology

G.Srivarun-2520030324

P.Teja-2520030569

Revanth-2520030571

Yashwanth-2520030581

1. INTRODUCTION

- Prime numbers are positive integers greater than 1 that have exactly two factors: 1 and the number itself.
- For very large integers, checking every possible divisor becomes computationally expensive and can lead to slow response times.
- The proposed web service provides a simple interface where users can submit a large integer and receive a prime/composite result.
- The project combines web technologies with efficient number-theory algorithms to demonstrate practical algorithm optimization.

2. PROBLEM STATEMENT

- Basic trial division checks possible divisors one by one, which becomes inefficient as the input size increases.
- A web-based system should provide quick feedback without making the user understand the mathematical process behind the validation.
- The system must validate user input, handle large integer values safely, and provide a clear result.
- The main challenge is balancing computational efficiency, correctness, usability, and scalability.

3. OBJECTIVES

- Create a clean and user-friendly web page for entering large integer values.
- Develop a reliable primality-validation module capable of processing significantly larger values than simple trial division.
- Apply modular arithmetic and an efficient primality-testing strategy to reduce unnecessary calculations.
- Clearly communicate whether the supplied value is prime, composite, or invalid.
- Design the system so that it can later be extended into an API or cryptographic learning platform.

4. PROPOSED SYSTEM

- The user enters a positive integer into the web interface.
- Client-side validation checks whether the input is numeric, within the supported range, and suitable for processing.
- The validated number is passed to the primality-testing module through the application logic.
- The algorithm performs modular exponentiation and primality checks to determine the result efficiently.
- The final status—Prime or Composite—is returned and displayed in an understandable format.

5. SYSTEM WORKFLOW

- Step 1 – Input: The user enters a large integer into the browser.
- Step 2 – Validation: The system checks the format and ensures the input is suitable for processing.
- Step 3 – Processing: The number is supplied to the primality-testing algorithm.
- Step 4 – Testing: Efficient modular arithmetic is used to perform the primality checks.
- Step 5 – Decision: The algorithm determines whether the number is prime or composite.
- Step 6 – Output: The result and any relevant validation message are displayed to the user.

INPUT

VALIDATE

TEST

DECIDE

RESULT

6. ALGORITHM

- Miller–Rabin is a fast primality test based on modular arithmetic and repeated modular exponentiation.
- First, the algorithm writes $n - 1$ as $d \times 2^s$, where d is odd, and then evaluates selected bases.
- For each base, modular powers are calculated to determine whether the number passes the required primality conditions.
- A number that fails a test is definitely composite; a number that passes is treated as a probable prime unless deterministic bases are used for the supported range.
- Compared with trial division, Miller–Rabin can process large integers much more efficiently and is widely used as a practical primality-testing technique.

7. TECHNOLOGIES USED

- HTML is used to structure the input form, buttons, result area, and other page elements.
- CSS provides a clean layout and improves readability and user experience.
- JavaScript can handle client-side validation, user interaction, and communication with the backend.
- A backend such as Python, Node.js, or Java can perform the large-integer primality calculation.
- The core mathematical concepts include modular arithmetic, modular exponentiation, and efficient primality testing.

8. EXPECTED OUTCOMES

- The completed system will allow users to enter large integers through an easy-to-use browser interface.
- It will identify prime and composite values using an efficient primality-testing approach.
- The solution will reduce unnecessary computation compared with checking every divisor individually.
- Results will be presented immediately and clearly, making the application suitable for demonstrations and learning.
- The project will illustrate how algorithm selection directly affects the performance of a software system.
- The design will provide a foundation for future extensions involving cryptography and secure key-generation concepts.

9. APPLICATIONS & FUTURE SCOPE

- Educational Tool: Demonstrates prime numbers, modular arithmetic, and algorithm efficiency in a practical application.
- Validation Service: Can be used to validate large integers in applications that require primality information.
- API Integration: The core validation logic can be exposed through an API for use by other applications.
- Cryptography Foundation: Prime-number validation is an important concept in public-key cryptography and can support further learning projects.
- Performance Enhancement: Future versions can include optimized big-integer libraries, caching, and asynchronous processing.
- Security Improvements: Production systems should use carefully selected deterministic/probabilistic testing methods and secure input handling.

10. CONCLUSION

- The project successfully connects web development with concepts from number theory and algorithm design.
- Using an efficient primality test avoids the excessive computation associated with straightforward divisor checking.
- The web interface makes the mathematical operation accessible to users without requiring them to run a program manually.
- The project demonstrates an important software-engineering principle: selecting an appropriate algorithm can significantly improve performance.
- With further development, the service can be extended into an API-based large-number utility or a cryptography-oriented educational platform.

THANK YOU

12

Questions & Answers

PROJECT TEAM

G. Srivarun | 2520030324

P. Teja | 2520030569

Revanth | 2520030571

Yashwanth | 2520030581